DATA 260 – Homework 9 Report

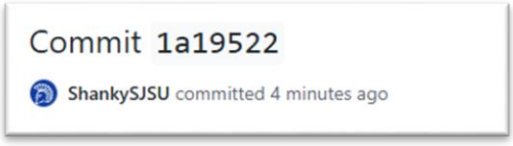
Student ID: 018322837 **Name: Shashank Ranjan**

Name	Definition	Value
SID	Student ID	018322837
SID4	Last 4 of Student ID	2837
PORT_BASE	8000 + SID mod 900(137)	8137
prefix	"s" + SID4	s2837
Seed	SID4	2837
verify sid	260000 + SID4	262837
DOMAIN_ID	SID4 MOD 8	5
Assigned Domain	Assignment domain	**Local restaurant inspections**

Repository: https://github.com/ShankySJSU/Data260-2837
Submission Tag: hw9 (how to create tag 😊

Please NOTE this could be different in the WORD document that you find on the repository, as this is source of the PDF file that I will be uploading to Canvas and the Report

The FINAL CODE's tag and repository is the following: (after this tag there WILL NOT BE any code change .. but only upload of the final Word /PDF document that you will see in PDF uploaded on CANVAS)
Tagged commit hash (hw9):



1a195225fae58225e97852360d5872288848baed

The above information is in the FINAL word/pDF document that you will find in the REPOSITORY.. because that is the base of all the code

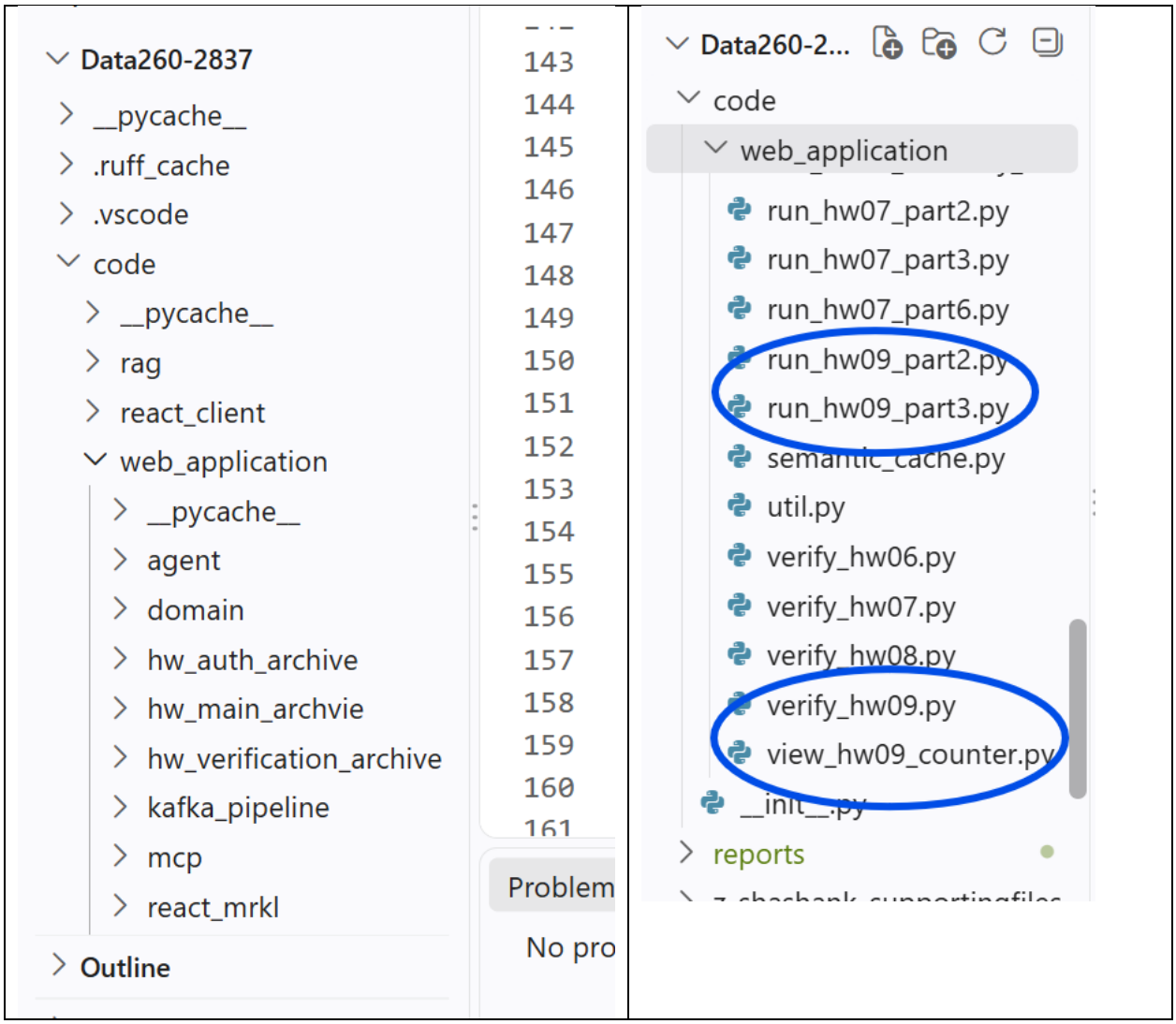
(Shashank: always use "git rev-parse HEAD"
Collaborators with access: Sbnikitha, supriyaselvanganesan

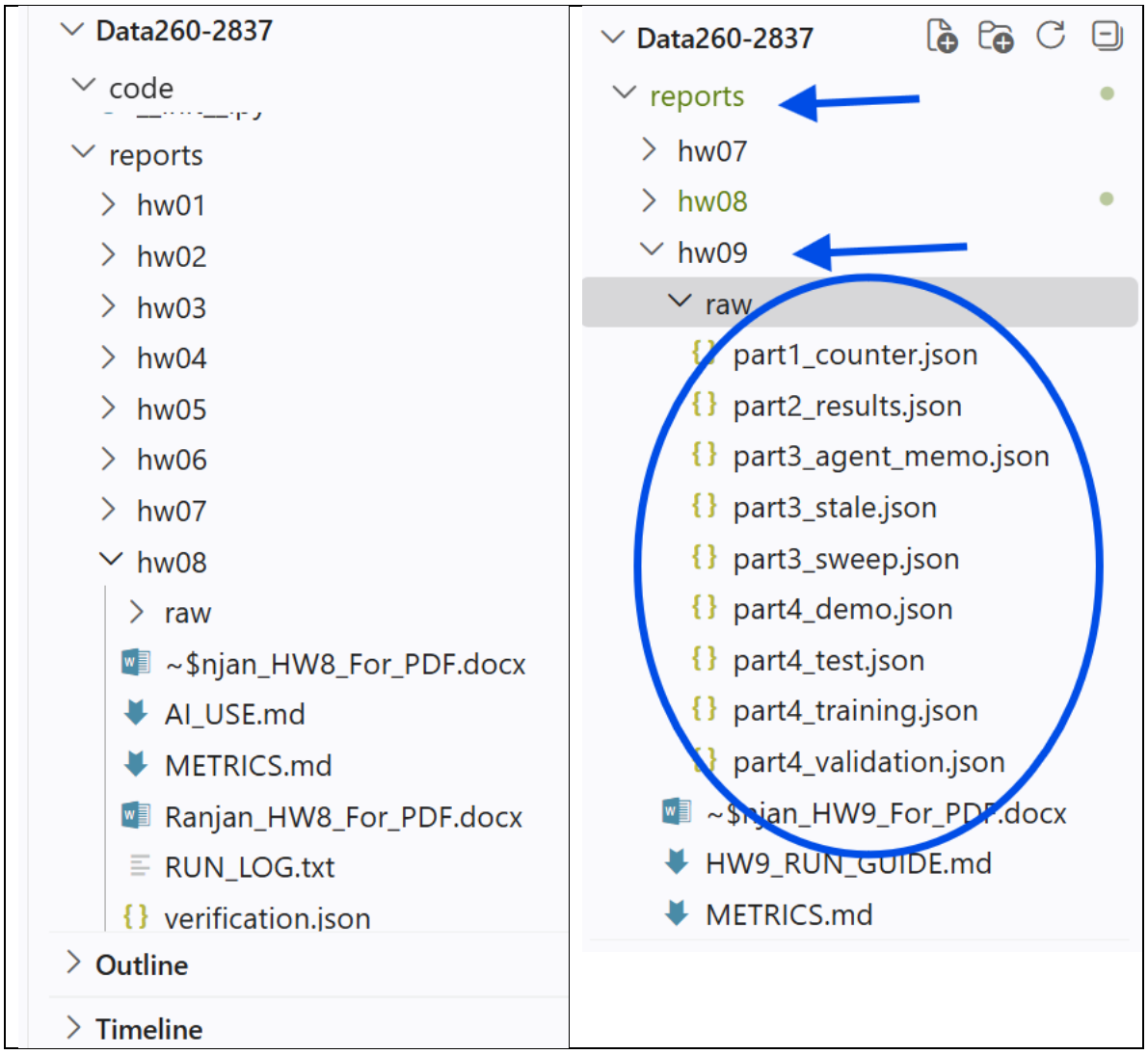
Hardware:
Local Windows PC, Python 3.12, VS Code Desktop, i7 CPU, 16GB RAM.

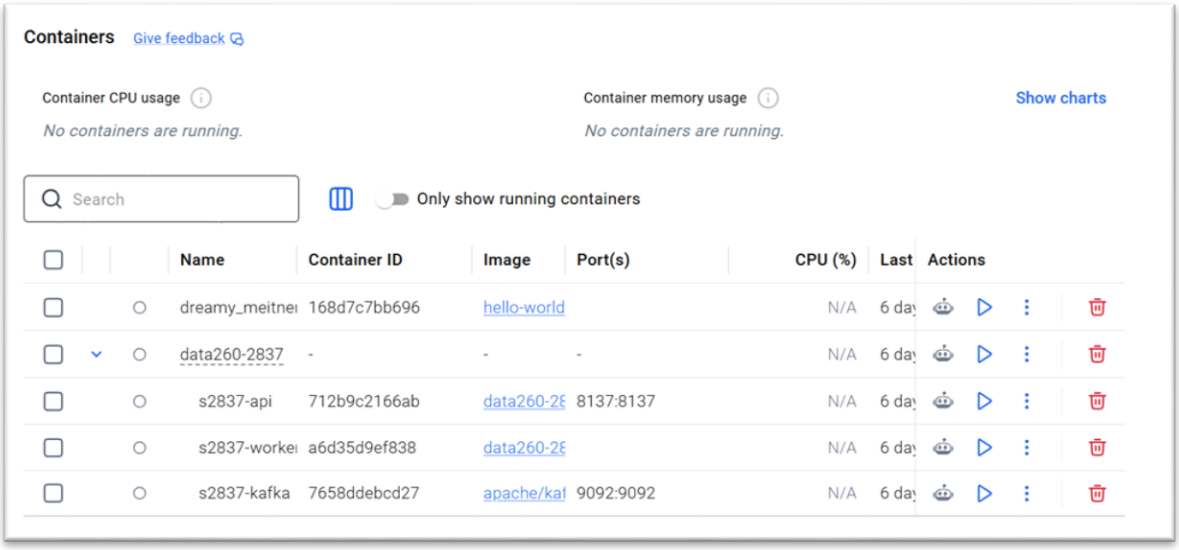
Model Used:

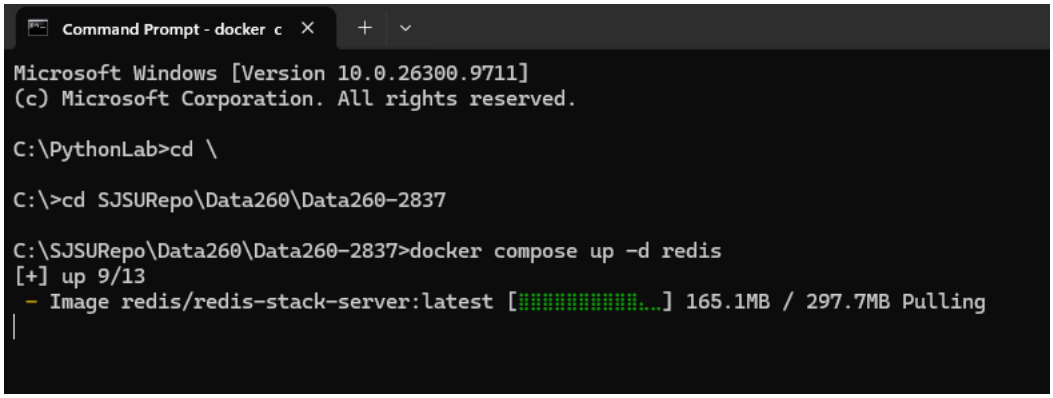
- Embeddings: sentence-transformers/all-MiniLM-L6-v2
- Ollama (qwen2.5:1.5b, qwen3:4b, llama3.2:1b)

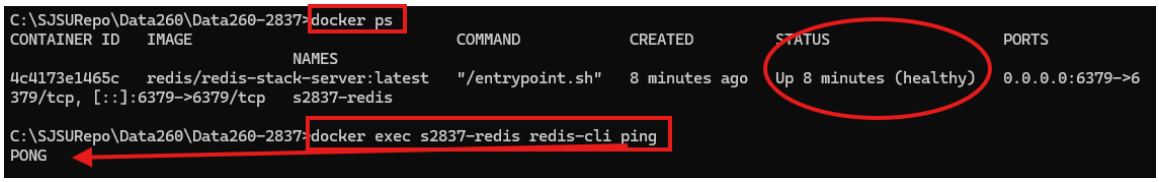
Base Directory Structure:




Data260-2837
code
reports
hw01
hw02
hw03
hw04
hw05
hw06
hw07
hw08
raw
~$njan_HW8_For_PDF.docx
AI_USE.md
METRICS.md
Ranjan_HW8_For_PDF.docx
RUN_LOG.txt
verification.json
Outline
Timeline
Data260-2837
reports
hw07
hw08
hw09
raw
part1_counter.json
part2_results.json
part3_agent_memo.json
part3_stale.json
part3_sweep.json
part4_demo.json
part4_test.json
part4_training.json
part4_validation.json
~$njan_HW9_For_PDF.docx
HW9_RUN_GUIDE.md
METRICS.md







Part 1 – Redis View Counter

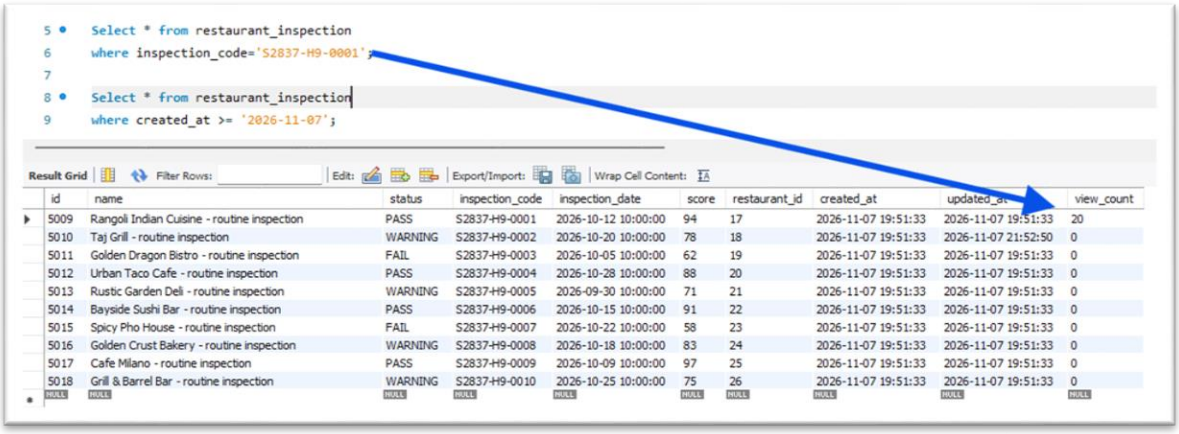
Every access to an inspection calls increment_inspection_views(inspection_id) on my InspectionStore (database + cache class). Redis INCR on s2837:views:inspection:<id> counts the view; every 10th increment writes the count to s2837_rel.restaurant_inspection.view_count (column added by HW9).

```
    def increment_inspection_views(self, inspection_id):
        """INCR the Redis counter; every SYNC_EVERY increments persist it to s2837_rel."""
        key = self.views_key(inspection_id)
        count = int(self.redis_client.incr(key))
        if count % cfg.SYNC_EVERY == 0:
            self.db.set_view_count(inspection_id, count)
            self.sync_log.append({"count": count, "db_value":
```

```
self.db.get_view_count(inspection_id)})
            self.last_synced = count
            log(COMPONENT, f"Synced views to database: {key} = {count} -> "
                           f"restaurant_inspection.view_count (id={inspection_id})",
echo=False)
        return count
```

```
============================
=== Testing Counter ===
Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
Access #10: Total = 10
✓ Synced views to database (view_count=10)
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
Access #20: Total = 20
✓ Synced views to database (view_count=20)
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25
2026-11-07 19:51:33.613 | Shashank Ranjan | s2837 | PART1      | final: Redis s2837:views:inspection:5009
= 25, s2837_rel.restaurant_inspection.view_count = 20 (last sync at a multiple of 10)
2026-11-07 19:51:33.702 | Shashank Ranjan | s2837 | PART1      | measured avg per view: Redis INCR path 0.
976 ms (includes the sync writes), direct DB UPDATE per view 3.38 ms
```

Measure	**Value**
Inspection	S2837-H9-0001 (id 5009)
Redis key	s2837:views:inspection:5009
Final Redis count	25
view_count in s2837_rel (last sync)	20
Avg ms per view, Redis INCR path	0.976
Avg ms per view if every view did a DB UPDATE	3.38



```
Select * from restaurant_inspection
where inspection_code='S2837-H9-0001';

Select * from restaurant_inspection
where created_at >= '2026-11-07';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	status	inspection_code	inspection_date	score	restaurant_id	created_at	updated_at	view_count
5009	Rangoli Indian Cuisine - routine inspection	PASS	S2837-H9-0001	2026-10-12 10:00:00	94	17	2026-11-07 19:51:33	2026-11-07 19:51:33	20
5010	Taj Grill - routine inspection	WARNING	S2837-H9-0002	2026-10-20 10:00:00	78	18	2026-11-07 19:51:33	2026-11-07 21:52:50	0
5011	Golden Dragon Bistro - routine inspection	FAIL	S2837-H9-0003	2026-10-05 10:00:00	62	19	2026-11-07 19:51:33	2026-11-07 19:51:33	0
5012	Urban Taco Cafe - routine inspection	PASS	S2837-H9-0004	2026-10-28 10:00:00	88	20	2026-11-07 19:51:33	2026-11-07 19:51:33	0
5013	Rustic Garden Deli - routine inspection	WARNING	S2837-H9-0005	2026-09-30 10:00:00	71	21	2026-11-07 19:51:33	2026-11-07 19:51:33	0
5014	Bayside Sushi Bar - routine inspection	PASS	S2837-H9-0006	2026-10-15 10:00:00	91	22	2026-11-07 19:51:33	2026-11-07 19:51:33	0
5015	Spicy Pho House - routine inspection	FAIL	S2837-H9-0007	2026-10-22 10:00:00	58	23	2026-11-07 19:51:33	2026-11-07 19:51:33	0
5016	Golden Crust Bakery - routine inspection	WARNING	S2837-H9-0008	2026-10-18 10:00:00	83	24	2026-11-07 19:51:33	2026-11-07 19:51:33	0
5017	Cafe Milano - routine inspection	PASS	S2837-H9-0009	2026-10-09 10:00:00	97	25	2026-11-07 19:51:33	2026-11-07 19:51:33	0
5018	Grill & Barrel Bar - routine inspection	WARNING	S2837-H9-0010	2026-10-25 10:00:00	75	26	2026-11-07 19:51:33	2026-11-07 19:51:33	0

Part 2 – Semantic Response Cache

The cache is the ingress of my Q&A endpoint (QAService.handle, also POST /qa in hw9_main.py). Each query is embedded with MiniLM and a KNN-1 search runs on the Redis vector index s2837:semcache:idx (FLAT, COSINE, 384 dims) over hashes s2837:semcache:entry:<id> that store query, embedding, final answer and a data-version marker. Similarity = 1 - cosine distance; threshold 0.85.

- Hit: return the stored final answer, is_cached=True, similarity, latency. The HW8 agents are not called.
- Miss: ground the question with the s2837_rel rows it mentions, run the HW8 pipeline (s2837_inbox -> planner -> writer -> reviewer -> s2837_final), store the entry, return is_cached=False.

```
def handle(self, query, threshold=None, validate_versions=True, use_cache=True):
    threshold = self.cache.threshold if threshold is None else threshold
    started_at = now_iso()
    t0 = time.perf_counter()
    result = {"query": query, "threshold": threshold, "started_at": started_at}

    vector = embed(query)
    entry, sim, embed_ms, search_ms = (self.cache.nearest(query, vector) if use_cache
                                       else (None, 0.0, 0.0, 0.0))
    result.update(similarity=sim, embed_ms=embed_ms, search_ms=search_ms,
                  nearest_query=entry["query"] if entry else None)

    stale = False
    if entry and sim >= threshold:
        if validate_versions:
            now_versions = self.cache.current_versions(list(entry["deps"]))
            changed = {c: (v, now_versions.get(c)) for c, v in entry["deps"].items()
                       if now_versions.get(c) != v}
            if changed:
                stale = True
                self.cache.delete(entry["key"])
                self.cache.bump_stat("stale_invalidations")
```

```
                    log(COMPONENT, f"STALE entry {entry['key']} similarity={sim:.4f} - data
"
                                   f"version changed {changed} -> invalidated, running
agent")
            if not stale:
                latency = ms_since(t0)
                self.cache.bump_stat("hits")
                log(COMPONENT, f"CACHE HIT similarity={sim:.4f} >= {threshold}
latency={latency} "
                               f"ms query={query!r} matched={entry['query']!r}")
                result.update(answer=entry["answer"], is_cached=True,
matched_query=entry["query"],
                              cache_key=entry["key"], latency_ms=latency, agent_ms=0.0,
                              stale_detected=False, finished_at=now_iso())
                return result

        # ---- miss: normal HW8 agent path ----
        log(COMPONENT, f"CACHE MISS best_similarity={sim:.4f} < {threshold}"
                       f"{' (stale entry removed)' if stale else ''} query={query!r}")
        self.cache.bump_stat("misses")
        records_codes = [r["inspection_code"] for r in self.db.hw9_records()
                         if r["restaurant"].lower() in query.lower()]
        versions = self.cache.current_versions(records_codes)  # read BEFORE running the
agent
        agent = run_agent(self.db, query, self.agent_mode)
        key = None
        if use_cache:
            t_store = time.perf_counter()
            key = self.cache.put(query, agent["answer"], versions, agent["agent_ms"],
                                 agent["mode"], vector)
            result["store_ms"] = ms_since(t_store)
        latency = ms_since(t0)
        log(COMPONENT, f"stored {key} deps={versions} total latency={latency} ms")
        result.update(answer=agent["answer"], is_cached=False, matched_query=None,
                      cache_key=key, latency_ms=latency, agent_ms=agent["agent_ms"],
                      agent_status=agent["status"], deps=versions, stale_detected=stale,
                      finished_at=now_iso())
        return result
```

#	Kind	is_cached	Similarity	Latency (ms)	Query
1	new	False	0.000	24753.1	What score did Taj Grill get in its latest inspection?
2	exact duplicate of #1	True	1.000	19.9	What score did Taj Grill get in its latest inspection?
3	paraphrase of #1	True	0.959	22.7	What was Taj Grill's most recent inspection score?
4	new	False	0.311	34778.7	What violations were found at Golden Dragon

					Bistro?
5	paraphrase of #4	True	0.870	18.8	Which violations did inspectors record at Golden Dragon Bistro?
6	new	False	0.320	21432.4	When must Rustic Garden Deli be re-inspected?
7	exact duplicate of #6	True	1.000	18.2	When must Rustic Garden Deli be re-inspected?
8	new	False	0.229	19305.9	What temperature must cold food be held at in a restaurant?
9	paraphrase of #8	True	0.914	16.1	At what temperature should a restaurant keep cold food?
10	new	False	0.533	42805.8	How often are restaurants inspected in our county?
11	exact duplicate of #4	True	1.000	21.7	What violations were found at Golden Dragon Bistro?
12	new (similar wording to #1, different restaurant)	False	0.581	17273.5	What score did Cafe Milano get in its latest inspection?

Metric	**Measured**
Queries	12
Cache hit rate	50.0%
Avg latency, cached	19.6 ms
Avg latency, non-cached (agent run)	26724.9 ms
Miss path	kafka
Threshold	0.85

These are the latencies I measured on my machine; the non-cached time is dominated by the two local model calls in the HW8 pipeline, while a hit costs one embedding plus one Redis KNN query.

- Planner listening

```
C:\SJSURepo\Data260\Data260-2837\code\web_application\kafka_pipeline>python planner.py
2026-11-07 20:23:32.110 | Shashank Ranjan | s2837 | PLANNER-1  | topic ready s2837_inspection_requests partitions=1
2026-11-07 20:23:32.112 | Shashank Ranjan | s2837 | PLANNER-1  | topic ready s2837_inspection_responses partitions=1
2026-11-07 20:23:32.112 | Shashank Ranjan | s2837 | PLANNER-1  | topic ready s2837_inbox partitions=1
2026-11-07 20:23:32.113 | Shashank Ranjan | s2837 | PLANNER-1  | topic ready s2837_tasks partitions=2
2026-11-07 20:23:32.114 | Shashank Ranjan | s2837 | PLANNER-1  | topic ready s2837_drafts partitions=1
2026-11-07 20:23:32.115 | Shashank Ranjan | s2837 | PLANNER-1  | topic ready s2837_final partitions=1
2026-11-07 20:23:32.115 | Shashank Ranjan | s2837 | PLANNER-1  | topic ready s2837_dead_letter partitions=1
2026-11-07 20:23:32.132 | Shashank Ranjan | s2837 | PLANNER-1  | planner started pid=6716 topic=s2837_inbox group=s2837_plan
ners idempotency=True commit_mode=after dlq=s2837_dead_letter max_retries=3
2026-11-07 20:23:32.418 | Shashank Ranjan | s2837 | PLANNER-1  | READY assigned partitions=[0] positions={0: 1} - Consumer i
s ready and listening...
2026-11-07 20:23:32.419 | Shashank Ranjan | s2837 | PLANNER-1  | IGNORED s2837_inbox[p0@1] test_run=verify-cc03bfa2 (this ag
ent handles test_run=None)
2026-11-07 20:23:32.438 | Shashank Ranjan | s2837 | PLANNER-1  | COMMITTED offset 2 on p0 after ignored
```

- Writer

```
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\kafka_pipeline> python writer.py
2026-11-07 20:25:24.422 | Shashank Ranjan | s2837 | WRITER-1   | topic ready s2837_inspection_requests partitions=1
2026-11-07 20:25:24.423 | Shashank Ranjan | s2837 | WRITER-1   | topic ready s2837_inspection_responses partitions=1
2026-11-07 20:25:24.424 | Shashank Ranjan | s2837 | WRITER-1   | topic ready s2837_inbox partitions=1
2026-11-07 20:25:24.424 | Shashank Ranjan | s2837 | WRITER-1   | topic ready s2837_tasks partitions=2
2026-11-07 20:25:24.425 | Shashank Ranjan | s2837 | WRITER-1   | topic ready s2837_drafts partitions=1
2026-11-07 20:25:24.425 | Shashank Ranjan | s2837 | WRITER-1   | topic ready s2837_final partitions=1
2026-11-07 20:25:24.426 | Shashank Ranjan | s2837 | WRITER-1   | topic ready s2837_dead_letter partitions=1
2026-11-07 20:25:24.443 | Shashank Ranjan | s2837 | WRITER-1   | writer started pid=15836 topic=s2837_tasks group=s2837_writ
ers idempotency=True commit_mode=after dlq=s2837_dead_letter max_retries=3
2026-11-07 20:25:24.689 | Shashank Ranjan | s2837 | WRITER-1   | READY assigned partitions=[0, 1] positions={0: 1, 1: 0} - C
onsumer is ready and listening...
```

- Reviewer

```
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\kafka_pipeline> python reviewr.py
C:\Python\python3.12\python.exe: can't open file 'C:\\SJSURepo\\Data260\\Data260-2837\\code\\web_application\\kafka_pipeline
\\reviewr.py': [Errno 2] No such file or directory
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\kafka_pipeline> python reviewer.py
2026-11-07 20:26:07.737 | Shashank Ranjan | s2837 | REVIEWER-1 | topic ready s2837_inspection_requests partitions=1
2026-11-07 20:26:07.738 | Shashank Ranjan | s2837 | REVIEWER-1 | topic ready s2837_inspection_responses partitions=1
2026-11-07 20:26:07.739 | Shashank Ranjan | s2837 | REVIEWER-1 | topic ready s2837_inbox partitions=1
2026-11-07 20:26:07.740 | Shashank Ranjan | s2837 | REVIEWER-1 | topic ready s2837_tasks partitions=2
2026-11-07 20:26:07.740 | Shashank Ranjan | s2837 | REVIEWER-1 | topic ready s2837_drafts partitions=1
2026-11-07 20:26:07.741 | Shashank Ranjan | s2837 | REVIEWER-1 | topic ready s2837_final partitions=1
2026-11-07 20:26:07.741 | Shashank Ranjan | s2837 | REVIEWER-1 | topic ready s2837_dead_letter partitions=1
2026-11-07 20:26:07.756 | Shashank Ranjan | s2837 | REVIEWER-1 | reviewer started pid=20368 topic=s2837_drafts group=s2837_r
eviewers idempotency=True commit_mode=after dlq=s2837_dead_letter max_retries=3
2026-11-07 20:26:07.989 | Shashank Ranjan | s2837 | REVIEWER-1 | READY assigned partitions=[0] positions={0: 1} - Consumer i
s ready and listening...
```

Code execution:

```
PS C:\SJSURepo\Data260\Data260-2837\code> python -m web_application.run_hw09_part2
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
er downloads.
Loading weights: 100%|████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 2309.92it/s]
2026-11-07 20:45:35.845 | Shashank Ranjan | s2837 | PART2      | ============================================================
===========
2026-11-07 20:45:35.846 | Shashank Ranjan | s2837 | PART2      | HW9 Part 2 - Shashank Ranjan - semantic cache threshold 0.8
5, embeddings sentence-transformers/all-MiniLM-L6-v2, miss path = HW8 agents (kafka); cleared 6 old entries
2026-11-07 20:45:35.848 | Shashank Ranjan | s2837 | PART2      | ============================================================
===========
2026-11-07 20:45:35.849 | Shashank Ranjan | s2837 | PART2      | --- query #1 [new] 'What score did Taj Grill get in its lat
est inspection?'
2026-11-07 20:45:35.872 | Shashank Ranjan | s2837 | QA         | CACHE MISS best_similarity=0.0000 < 0.85 query='What score
did Taj Grill get in its latest inspection?'
2026-11-07 20:45:36.164 | Shashank Ranjan | s2837 | SENDER     | SENT s2837_inbox[p0@8] correlation_id=a5898ca7-33f9-4539-b0
77-fa1b6c33caa0 question='What score did Taj Grill get in its latest inspection?\n\nFacts from s2837_rel (use them exactly):
```

Here is the Reports:

```
2026-11-07 20:48:16.364 | Shashank Ranjan | s2837 | PART2      | RESULTS
2026-11-07 20:48:16.366 | Shashank Ranjan | s2837 | PART2      | ============================================================
===========
 # kind                       cached    sim  latency ms  query
 1 new                        False   0.000     24753.1  What score did Taj Grill get in its latest inspection?
 2 exact duplicate of #1      True    1.000        19.9  What score did Taj Grill get in its latest inspection?
 3 paraphrase of #1           True    0.959        22.7  What was Taj Grill's most recent inspection score?
 4 new                        False   0.311     34778.7  What violations were found at Golden Dragon Bistro?
 5 paraphrase of #4           True    0.870        18.8  Which violations did inspectors record at Golden Dragon Bistro?
 6 new                        False   0.320     21432.4  When must Rustic Garden Deli be re-inspected?
 7 exact duplicate of #6      True    1.000        18.2  When must Rustic Garden Deli be re-inspected?
 8 new                        False   0.229     19305.9  What temperature must cold food be held at in a restaurant?
 9 paraphrase of #8           True    0.914        16.1  At what temperature should a restaurant keep cold food?
10 new                        False   0.533     42805.8  How often are restaurants inspected in our county?
11 exact duplicate of #4      True    1.000        21.7  What violations were found at Golden Dragon Bistro?
12 new (similar wording to #1 False   0.581     17273.5  What score did Cafe Milano get in its latest inspection?
2026-11-07 20:48:16.368 | Shashank Ranjan | s2837 | PART2      | hit rate 50.0% (6/12); avg latency cached 19.6 ms vs non-ca
ched 26724.9 ms
2026-11-07 20:48:16.371 | Shashank Ranjan | s2837 | PART2      | saved C:\SJSURepo\Data260\Data260-2837\reports\hw09\raw\par
t2_results.json
PS C:\SJSURepo\Data260\Data260-2837\code>
```

Part 3 – Threshold Economics and Stale Answers

3.1 Labeled test set

code/web_application/hw9/cache_cases.yaml has 40 domain queries: 10 exact duplicates, 10 paraphrases (should hit), 10 superficially similar questions with different correct answers and 10 unrelated questions (should miss). The cache is warmed with the 30 cached_query entries; every query is looked up against all of them. The file was committed before the sweep and not relabeled (errata go to cache_cases_errata.md).

```
- {id: S01, category: similar_different, expected: miss,
   cached_query: "What score did Bayside Sushi Bar get in its latest inspection?",
   query: "What score did Golden Crust Bakery get in its latest inspection?",
   why_different: "different restaurant: 91 vs 83"}
- {id: P03, category: paraphrase, expected: hit,
   cached_query: "When is Spicy Pho House's re-inspection deadline?",
   query: "By what date does Spicy Pho House have to be inspected again?"}
```

3.2 Threshold sweep

Hit rate = hits / 40. False-hit rate = false hits / 40 (a hit on a should-miss query, or a paraphrase hitting the wrong entry). Mean latency over the 40 queries: hits use the measured lookup time, misses add the measured agent time for that query (each query answered once by the HW8 agents and reused across thresholds).

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	70.0%	22.5%	7537.9	$18.24
0.80	60.0%	15.0%	10087.0	$12.32
0.85	57.5%	15.0%	10785.7	$12.34
0.90	45.0%	10.0%	13864.7	$8.44
0.95	35.0%	5.0%	15782.3	$4.52

Category	min sim	mean sim	max sim
exact_duplicate	1.0	1.0	1.0
paraphrase	0.7547	0.8854	0.9694
similar_different	0.708	0.8423	0.9623
unrelated	0.3642	0.4762	0.7479

3.3 One false hit in full

Field	Value
Threshold	0.85
Case	S10 (similar_different)
Cached query	Is a score of 84 a PASS?
Incoming query	Is a score of 85 a PASS?
Similarity	0.9623
Cached answer returned	1. The score range for a PASS is 85-100. 2. The given score is 84, which falls within the range of 85-100. 3. Therefore, a score of 84 is a PASS. The restaurant does not need to be re-inspected within the specified time frame.
Correct answer	Based on the provided facts, a score of 85 falls within the PASS range (85-100). According to the county policy, a score of 85-100 indicates that the restaurant is in compliance and no re-inspection is required. Therefore, a score of 85 is a PASS.
Records involved	-

Why the embedding confused them: the two questions share 5 words (a, is, of, pass, score) and differ only in 84, 85 (word Jaccard 0.714). MiniLM mean-pools all token vectors, so the shared question template dominates the sentence vector while the few tokens that change the answer (a restaurant name, a status word or a number) move it only slightly. The cosine similarity therefore stays at 0.9623, above the threshold, although the correct answers differ (84 = WARNING (no) vs 85 = PASS (yes)).

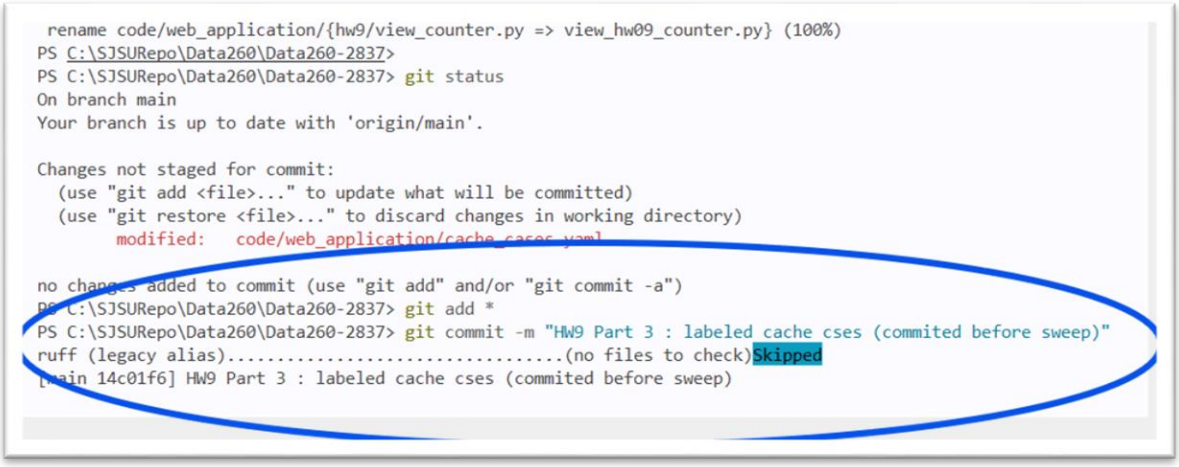
3.4 Cost model and threshold choice

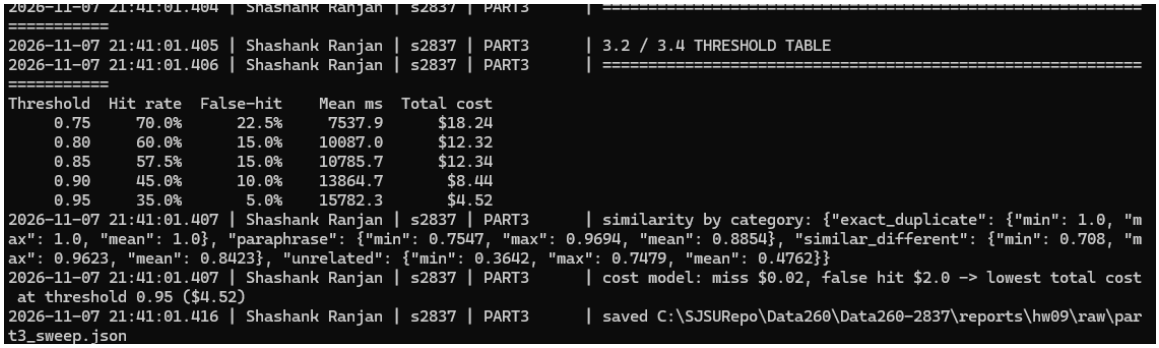
Outcome	Cost	Justification
Cache miss	$0.02	A miss runs the full HW8 Planner->Writer->Reviewer pipeline: two model calls of ~1-2k tokens plus several seconds of user waiting. Priced like a hosted small model call plus the user's waiting time: $0.02.
False hit	$2.0	A false hit shows a wrong score/status/date for a real restaurant (e.g. 'Golden Crust Bakery scored 91' when it scored 83). A diner or owner acts on it or contacts support: ~6 minutes of staff time at $20/h = $2.00,

		ignoring reputational/legal risk. So one false hit costs as much as 100 misses.

Total cost = misses x miss cost + false hits x false-hit cost, over the 40 queries (a correct hit costs ~0).

Threshold to ship: 0.95. It has the lowest total cost ($4.52) with hit rate 35.0% and false-hit rate 5.0%. Because one false hit costs as much as 100 misses, the cost curve is driven by false hits: lower thresholds buy a few extra paraphrase hits but pay for them with wrong restaurant facts.





3.5 Stale answers and invalidation

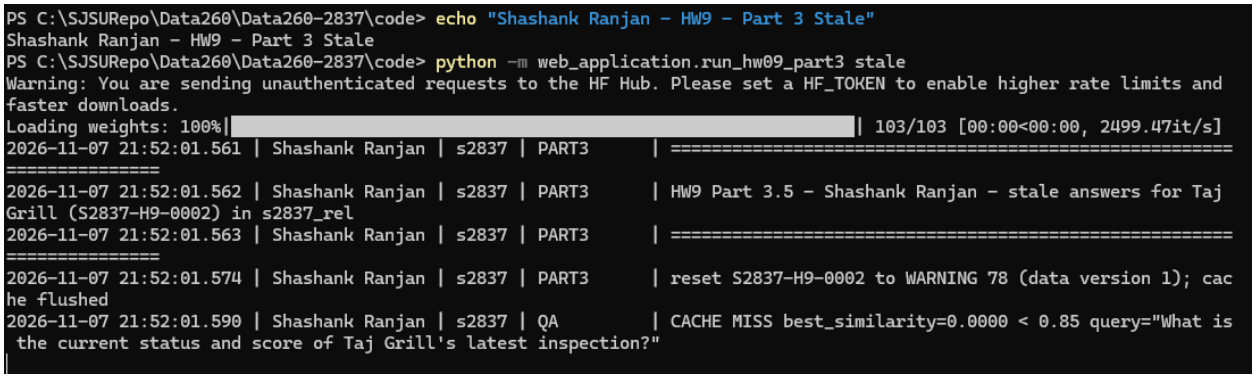
Query: "What is the current status and score of Taj Grill's latest inspection?" depends on s2837_rel record S2837-H9-0002 (Taj Grill). When the answer is stored, the entry keeps deps = {inspection_code: version} read from s2837:dv:inspection:<code>. db.update_inspection() is the write path: it updates s2837_rel and INCRs that marker. On every hit the service compares the stored versions with the live ones; a mismatch deletes the entry and reruns the agents.

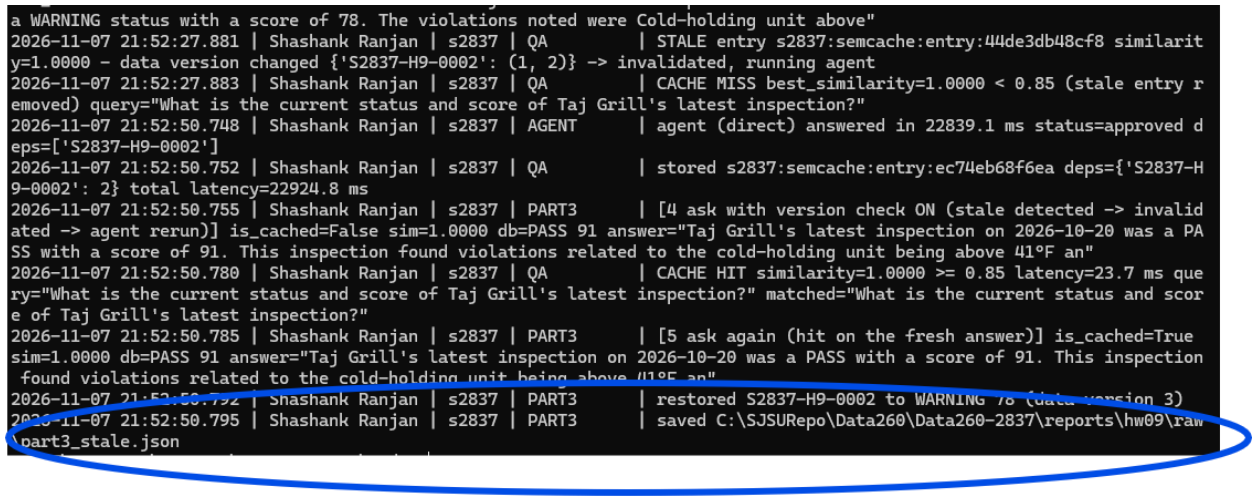
Step	is_cached	Similarity	s2837_rel now	Answer (start)
1 first ask (miss, stored with data-	False	0.0	WARNING 78	Taj Grill's latest inspection on

version marker)				2026-10-20 was conducted and resulted in a WARNING status with a score of 78. The violations noted were Cold-holding unit above...
2 ask again (hit, still correct)	True	1.0	WARNING 78	Taj Grill's latest inspection on 2026-10-20 was conducted and resulted in a WARNING status with a score of 78. The violations noted were Cold-holding unit above...
3 ask with version check OFF (STALE HIT - old answer)	True	1.0	PASS 91	Taj Grill's latest inspection on 2026-10-20 was conducted and resulted in a WARNING status with a score of 78. The violations noted were Cold-holding unit above...
4 ask with version check ON (stale detected -> invalidated -> agent rerun)	False	1.0	PASS 91	Taj Grill's latest inspection on 2026-10-20 was a PASS with a score of 91. This inspection found violations related to the cold-holding unit being above 41°F an...
5 ask again (hit on the fresh answer)	True	1.0	PASS 91	Taj Grill's latest inspection on 2026-10-20 was a PASS with a score of 91. This inspection found violations related to the cold-holding unit being above 41°F an...

Trade-off: the check costs one extra Redis MGET per hit and only reacts to writes that go through update_inspection (a direct SQL update would not bump the marker, so the 24 h TTL remains as a safety net). In exchange, invalidation is precise: only answers that used the changed record are dropped, and the rest of the cache keeps its hit rate. Answers about general rules have no deps and are never invalidated by record changes.

PS C:\SJSURepo\Data260\Data260-2837\code>python -m web_application.run_hw09_part3 stale





Part 4 – Self-Improving Multi-Agent System

Answering Agent: local Ollama qwen2.5:1.5b (HW1 adapter). It looks up the s2837_rel row of the restaurant named (restaurant, code, date, status, score, violations) and loads the lessons for the question type. Evaluator Agent: checks the answer against reference_data.json (ground truth incl. derived re-inspection dates and county score bands) and returns PASS/REVISE, a short explanation and correction feedback.

File	Content
reference_data.json	10 restaurants/inspections + county rules (Evaluator

	only)
training_data.json	10 questions - used to create lessons
validation_data.json	10 questions - decides which lessons are kept
test_data.json	12 held-out questions - final evaluation only
lessons.json	1 lessons with lesson_id, failure_pattern, correction, created_at, use_count

```
    def run(self, qid, question, max_attempts=MAX_ATTEMPTS, allowed_lessons=None,
            use_lessons=False, controlled_bad=False):
        start = time.perf_counter()
        qtype = classify(question)
        lessons = []
        if use_lessons and self.store is not None:
            lessons = self.store.relevant(qtype, allowed_lessons)
            for x in lessons:
                log(COMPONENT, f"LESSON LOADED {x['lesson_id']} ({x['failure_code']}) for
{qid} "
                               f"- use_count now {x['use_count']}")
        attempts, previous, feedback = [], None, None
        for n in range(1, max_attempts + 1):
            ans = self.answerer.answer(question, lessons, previous, feedback,
                                       controlled_bad=controlled_bad and n == 1)
            ev = self.evaluator.evaluate(question, ans["answer"])
            attempts.append({"attempt": n, "answer": ans["answer"], "model": ans["model"],
                             "answer_ms": ans["latency_ms"], "evaluation": ev,
                             "feedback_given": feedback})
            log(COMPONENT, f"{qid} attempt {n}: {ev['result']} - {ev['explanation']}"
                           + (f" | feedback: {ev['feedback']}" if ev["feedback"] else ""))
            if ev["result"] == "PASS":
                break
            previous, feedback = ans["answer"], ev["feedback"]
        final = attempts[-1]
        return {"id": qid, "question": question, "question_type": qtype,
                "lessons_loaded": [x["lesson_id"] for x in lessons], "attempts": attempts,
                "n_attempts": len(attempts), "final_answer": final["answer"],
                "final_result": final["evaluation"]["result"],
                "first_failure_codes": attempts[0]["evaluation"]["failure_codes"],
                "latency_ms": round((time.perf_counter() - start) * 1000, 1)}

    def evaluate(self, question, answer):
        text = (answer or "").lower()
        qtype = classify(question, self.ref)
        rec = find_restaurant(question, self.ref)
        codes, checks, fixes = [], {}, []

        if qtype in ("score", "status"):
            checks["score"] = _num(text, rec["score"])
            if not checks["score"]:
                codes.append("MISSING_SCORE")
                fixes.append(f"state the score: {rec['score']}")
        if qtype == "status":
            right = rec["status"].lower() in text
            wrong = [s for s in STATUSES if s != rec["status"] and s.lower() in text]
            checks["status"] = right
            if not right:
```

```
            codes.append("WRONG_STATUS" if wrong else "MISSING_STATUS")
            fixes.append(f"the status is {rec['status']}")
    if qtype == "reinspection":
        if rec["reinspection_due"]:
            ok = any(v in text for v in date_variants(rec["reinspection_due"]))
            checks["due_date"] = ok
            if not ok:
                days = self.ref["rules"]["reinspection_days"][rec["status"]]
                codes.append("MISSING_REINSPECTION")
                fixes.append(f"{rec['restaurant']} is {rec['status']} (inspected "
                             f"{rec['inspection_date']}); {rec['status']} requires "
                             f"re-inspection within {days} days, so it is due by "
                             f"{rec['reinspection_due']}")
        else:
# ... (see repository for the rest)
```

4.1 Revision sequence (controlled test case)

Field	Value
Question	What score did Rangoli Indian Cuisine receive in its latest inspection?
Initial answer (intentionally incomplete)	Rangoli Indian Cuisine had a routine health inspection recently.
Evaluator result	REVISE - score question; failed: MISSING_SCORE
Evaluator feedback	Fix: state the score: 94
Revised answer	Rangoli Indian Cuisine received a score of 94 in its latest inspection.
Final evaluation	PASS - score question; all required facts present and supported
Number of attempts	2
Final response	Rangoli Indian Cuisine received a score of 94 in its latest inspection.

```
2026-11-07 22:20:45.982 | Shashank Ranjan | s2837 | PART4      | ================================================
==============
Variant                 Correct Incorrect  Avg attempts    Avg ms
Baseline                      8         4           1.0    6735.7
Evaluator Repair             11         1          1.33    7370.0
Lesson Memory                11         1          1.25    7719.5
Validated Improvement        11         1          1.33    7752.5
2026-11-07 22:20:45.988 | Shashank Ranjan | s2837 | PART4      | ================================================
==============
2026-11-07 22:20:45.989 | Shashank Ranjan | s2837 | PART4      | HW9 Part 4.1 CONTROLLED REVISION DEMO (first answer int
entionally incomplete)
2026-11-07 22:20:45.990 | Shashank Ranjan | s2837 | PART4      | ================================================
==============
2026-11-07 22:20:45.992 | Shashank Ranjan | s2837 | ANSWERER   | ANSWER (controlled-incomplete (no model call), 0.4 ms,
lessons=[]): 'Rangoli Indian Cuisine had a routine health inspection recently.'
2026-11-07 22:20:45.992 | Shashank Ranjan | s2837 | SYSTEM     | demo_controlled attempt 1: REVISE - score question; fai
led: MISSING_SCORE | feedback: Fix: state the score: 94
2026-11-07 22:20:50.877 | Shashank Ranjan | s2837 | ANSWERER   | REVISED (ollama:qwen2.5:1.5b, 4883.3 ms, lessons=[]): '
Rangoli Indian Cuisine received a score of 94 in its latest inspection.'
2026-11-07 22:20:50.881 | Shashank Ranjan | s2837 | SYSTEM     | demo_controlled attempt 2: PASS - score question; all r
equired facts present and supported
2026-11-07 22:20:50.886 | Shashank Ranjan | s2837 | PART4      | final: PASS after 2 attempts -> 'Rangoli Indian Cuisine
 received a score of 94 in its latest inspection.'
2026-11-07 22:20:50.901 | Shashank Ranjan | s2837 | PART4      | 4.2 lesson lesson_001 created after failed train_03 ('B
y what date does Golden Dragon Bistro need its follow-up inspection?') and LOADED for later train_07 ('What is the re-in
spection deadline for Spicy Pho House?') -> PASS in 2 attempt(s)
PS C:\SJSURepo\Data260\Data260-2837\code>
```

4.2 Lesson saved and reused

Field	Value
Lesson	lesson_001 (MISSING_REINSPECTION)
Created after failed answer	train_03: By what date does Golden Dragon Bistro need its follow-up inspection?
Failed initial answer	Golden Dragon Bistro needs its follow-up inspection on 2027-04-05.
failure_pattern	Answers about re-inspection omit the due date (or imply one for a PASS).
correction	Compute it from the record: WARNING -> re-inspect within 30 days of the inspection date, FAIL -> within 10 days, PASS -> no re-inspection is required. State the exact due date as YYYY-MM-DD.
Later question that loaded it	train_07: What is the re-inspection deadline for Spicy Pho House?
Answer with lesson	The re-inspection deadline for Spicy Pho House is 2026-11-01.
Result	PASS in 2 attempt(s)


```
{
 "lesson_id": "lesson_001",
 "failure_pattern": "Answers about re-inspection omit the due date (or imply one for a PASS).",
 "correction": "Compute it from the record: WARNING -> re-inspect within 30 days of the inspection date, FAIL -> within 10 days, PASS -> no re-inspection is required. State the exact due date as YYYY-MM-DD.",
 "created_at": "2026-11-07",
 "use_count": 7
}
```

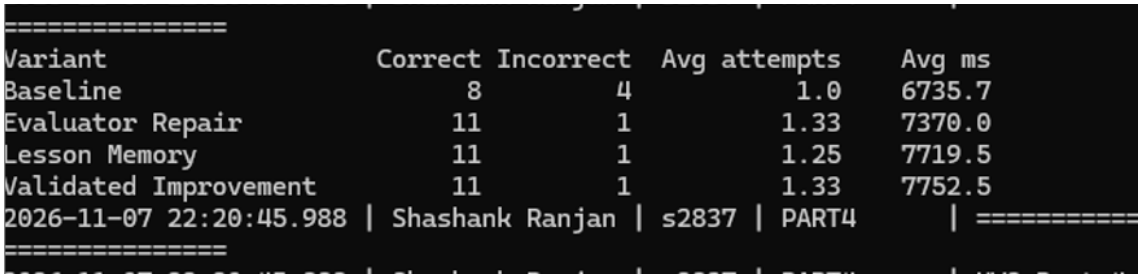

4.3 Validation before keeping a lesson

Lesson	Code	Decision	Validation correct (before -> after)	First-try passes	Regressions	Unsupported answers
lesson_001	MISSING_REINSPECTION	rejected	8 -> 8	5 -> 6	none	val_06

Keep rule: failure cases improve or stay the same, previously correct answers stay correct, and questions about restaurants without a record get no unsupported answer. Lessons are tried one at a time on top of the already kept ones.

4.4 Four system variants on test_data.json

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	8	4	1.0	6735.7
Evaluator Repair	11	1	1.33	7370.0
Lesson Memory	11	1	1.25	7719.5
Validated Improvement	11	1	1.33	7752.5



4.5 Explanation

- Agent communication: the orchestrator passes the question to the Answering Agent, sends {question, answer} to the Evaluator, and on REVISE returns {previous answer, feedback} to the Answering Agent.
- Lesson reuse: a lesson is created when the Evaluator reports the same failure code on two training answers; before answering, the Answering Agent loads lessons whose applies_to matches the question type and adds their corrections to its system prompt (use_count increments).
- Stopping condition: PASS or two answer attempts, whichever comes first.
- Validation: each candidate lesson is tested on validation_data.json against the current kept set and kept only if it causes no regression; test_data.json is used once per variant at the end and never creates lessons.
- Limitation: the Evaluator is a rule-based fact checker over reference_data.json, so it can only verify facts it knows how to find (scores, statuses, dates, violation keywords, score bands); a fluent answer with an extra unsupported detail outside these checks could still PASS. Also, because feedback contains the correct fact, Evaluator Repair mostly measures how well the model follows feedback.

verification.json (make verify-hw09)

24 of 24 checks passed.

Check	Status
redis_reachable_with_search_module	PASS
all_hw9_redis_keys_start_with_prefix	PASS
part1_counter_counts_1_to_25	PASS
part1_synced_every_10	PASS
part1_key_has_prefix	PASS
part2_at_least_10_queries_mixed	PASS
part2_rows_have_flag_similarity_latency	PASS
part2_exact_duplicates_hit	PASS
part2_summary_reported	PASS
cache_cases_40_labeled_10_per_category	PASS
sweep_five_thresholds	PASS
sweep_table_complete	PASS
false_hit_example_complete	PASS
threshold_chosen_by_cost	PASS

stale_hit_demonstrated	PASS
invalidation_by_data_version	PASS
input_datasets_contain_questions_only	PASS
lessons_json_required_fields	PASS
no_lesson_from_test_split	PASS
lessons_validated	PASS
four_variants_on_test	PASS
max_two_attempts	PASS
controlled_revision_sequence	PASS
lesson_reused_on_later_question	PASS

AI Use

1. What I used AI for and what I did myself.
I used Claude (Anthropic) to generate the first version of the HW9 code (Redis view counter, semantic cache, threshold sweep, stale-answer check, Answering/Evaluator agents and the report builder), and to explain the caching concepts. I set up Redis Stack in Docker and moved and renamed the files to fit my repository layout. I fixed the problems on my own machine: the relative-import errors after moving the files, the report builder's imports, and the HW8 agents falling back to template answers because I had moved adapter.py into an archive folder. I ran all four parts with my local MySQL, Redis, Kafka and Ollama, checked the database and Redis values by hand, and chose which results to report.

2. One AI-produced output that was wrong.
The Evaluator Agent the AI wrote recognized "no record" answers only from a fixed list of phrases. For test_06 ("What score did Blue Lotus Kitchen get?", a restaurant with no record), my model answered correctly: "Blue Lotus Kitchen did not have a restaurant health inspection recorded." The Evaluator still marked it UNSUPPORTED_CLAIM in all four variants.

3. How I detected it.
All four variants had exactly one wrong test answer, so I printed the failing questions together with their answers. Reading test_06's answer showed it was correct and the grading was wrong. Then I checked why lesson_001 had been rejected in validation. It had no regressions and improved first-try passes from 5 to 6, but it failed the "no unsupported answers" check because of val_06 (Harbor View Grill), the same kind of correct refusal.

4. What I changed and why.
I chose not to rerun the experiments and to report the results as measured. The bug removed one correct answer from every variant equally, so the comparison between variants is still fair. The fix is to match a negation followed by "record/inspection/data" instead of fixed phrases; I tested that it accepts this answer and still rejects invented scores. The lesson I take from it: an evaluator that rejects correct answers is as harmful as one that accepts wrong ones, because here it also blocked a useful lesson.